Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Program: Software Engineering

Operating Systems

Laboratory Work No. 2

Meet the OS You Will Build

Track B: Build Your Own Small OS (xv6)

Done by:
st. gr. FAF-241
Sitarciuc Dmitri

Checked by:
Reițman Patricia

Chișinău – 2026

Laboratory Work No. 2 – Meet the OS You Will Build

1. Introduction

The purpose of this laboratory work was to get familiar with xv6, a small Unix-like teaching operating system from MIT. I compiled xv6 from source, booted it in the QEMU RISC-V emulator, used its shell, read part of its source code and added my own user program, `sleep`.

The work was done in GitHub Codespaces (Ubuntu 24.04 in a cloud container), the same environment as in Lab 1. xv6 runs inside QEMU, so here there are two operating systems at once: Linux, where I compile the code, and xv6, which runs as a guest inside the emulator.

2. Build and Boot xv6

First I installed the RISC-V cross-compiler and QEMU, then downloaded the xv6 source:

```
sudo apt-get install -y build-essential gdb-multiarch qemu-system-misc gcc-
riscv64-linux-gnu binutils-riscv64-linux-gnu
git clone https://github.com/mit-pdos/xv6-riscv
cd ~/xv6-riscv
ls
qemu-system-riscv64 --version
```

```
LICENSE  Makefile  README  kernel  mkfs  test-xv6.py  user
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.18)
```

Figure 2.1 – xv6 source tree and installed QEMU version

The cross-compiler is needed because my computer is x86-64, while xv6 is built for the RISC-V processor. Then I built and started xv6:

```
make qemu
```

`make` compiled every file in `kernel/` and `user/` with `riscv64-linux-gnu-gcc`, linked the kernel, created the disk image `fs.img` with `mkfs`, and started QEMU:

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -
smp 3 -nographic ...

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

Figure 2.2 – QEMU command line and xv6 boot messages (QEMU options shortened)

2.1. Interesting Command Output

The QEMU command line shows the virtual machine xv6 gets: 128 MB of RAM (`-m 128M`) and 3 CPU cores (`-smp 3`). A RISC-V core is called a *hart*. Hart 0 boots the kernel and then wakes harts 1 and 2. On the second boot the order was `hart 2 starting` before `hart 1 starting` , which shows the cores really start in parallel and the order is not fixed. Finally the kernel starts the first process, `init` , which starts the shell `sh` with the `$` prompt.

3. Using xv6 as Unix

At the xv6 `$` prompt I ran several standard commands:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35408
echo           2 4 34344
forktest       2 5 17312
grep           2 6 42768
init           2 7 34800
kill           2 8 34272
ln             2 9 34064
ls             2 10 41552
mkdir          2 11 34312
rm             2 12 34296
sh             2 13 56168
...
wc             2 17 36368
sync           2 22 33744
console        3 23 0
```

```
$ echo hello xv6
hello xv6
$ ls | grep c
cat            2 3 35408
echo           2 4 34344
wc             2 17 36368
sync           2 22 33744
console        3 23 0
$ wc README
48 336 2441 README
```

Figure 3.1 – ls, echo, a pipe and wc in the xv6 shell (part of the ls listing omitted)

`cat README` printed the xv6 README file, which says that xv6 is a re-implementation of Unix Version 6 for RISC-V, written in ANSI C for MIT's 6.1810 course.

The columns of `ls` are: name, type (1 = directory, 2 = file, 3 = device), inode number and size in bytes. All programs are stored as files in the root directory `/`.

3.1. Observations

1. Three programs shipped with xv6: `cat`, `echo` and `wc`. Others are `ls`, `grep`, `mkdir`, `rm`, `kill` and `sh`.
2. For `ls | grep c` to work, the OS must support **processes** (the shell creates two of them with `fork` and starts `ls` and `grep` in them with `exec`) and **inter-process communication** (the `pipe` system call creates a kernel buffer: `ls` writes into one end and `grep` reads from the other).
3. The xv6 shell feels like a very small version of the Linux shell from Lab 1: running programs, pipes and redirection work the same way, but there are only about 20 programs and no features like command history or Tab completion.

3.2. Interesting Command Output

The pipe result was the most interesting. `grep c` kept only the lines that contain the letter c, including `echo` and `sync`, whose names contain c in the middle, and `console`, which is a device (type 3, size 0), not a program. Also, `wc README` reported 2441 bytes, exactly the size `ls` showed for `README`, so both programs see the same file through the kernel.

4. Reading the Source Code

After stopping QEMU, I looked at the source from Linux. The project has two main folders: `kernel/` (the operating system) and `user/` (ordinary programs).

```
sed -n '1,40p' user/ls.c
sed -n '1,30p' user/cat.c
grep -n "read\|write\|open\|close\|pause\|sleep" user/user.h
grep -n "sys_read\|sys_write" kernel/sysfile.c
```

The main loop of `user/cat.c`:

```
while ((n = read(fd, buf, sizeof(buf))) > 0) {
  if (write(1, buf, n) != n) {
    fprintf(2, "cat: write error\n");
    exit(1);
  }
}
```

Figure 4.1 – Main loop of user/cat.c

System call declarations in `user/user.h` and kernel implementations in `kernel/sysfile.c`:

```
10:int write(int, const void *, int);
11:int read(int, void *, int);
12:int close(int);
15:int open(const char *, int);
25:int pause(int);

69:sys_read(void)
83:sys_write(void)
```

Figure 4.2 – System call declarations in user/user.h and kernel implementations in kernel/sysfile.c

4.1. Observations

1. `cat` uses `read` to ask the kernel for up to 512 bytes from a file into `buf`, and `write(1, buf, n)` to ask the kernel to send them to file descriptor 1 (the screen). It also uses `open` and `close` to open and release files and `exit` to end the process. If no file is given, it reads from descriptor 0 (the keyboard).
2. `sys_read` is implemented in `kernel/sysfile.c` at line 69, and `sys_write` at line 83.
3. Code in `kernel/` is the OS itself and runs in privileged (supervisor) mode with full access to hardware. Code in `user/` runs in user mode and cannot touch hardware directly: it can only ask the kernel for services through system calls such as `read` and `write`.

4.2. Interesting Command Output

The `user/user.h` search found `int pause(int);` on line 25 but no `sleep`. In this version of xv6 the system call that waits for a number of timer ticks is named `pause`, not `sleep` as in older versions and in the lab sheet. This matters for the next part.

5. First xv6 Program: sleep

The task was to write a user program `sleep` that takes a number of clock ticks and waits that long.

5.1. Implementation

I created `user/sleep.c`:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));
  exit(0);
}
```

Figure 5.1 – Final version of user/sleep.c

`argc` is the number of words on the command line, including the program name, so `sleep 10` gives `argc = 2`. If the number is missing, the program prints a usage message to descriptor 2 (standard error) and exits with code 1. Otherwise `atoi` converts the text `"10"` to the integer 10 and `pause` asks the kernel to put the process to sleep for that many ticks. I used `pause()` because, as found in Part 4, this xv6 version declares `int pause(int);` and has no `sleep()` system call.

To include the program in the build, I added `$U/_sleep\` to the `UPROGS` list in the `Makefile`, right after `_ls`:

```
141:    $U/_ls\
142:    $U/_sleep\
```

Figure 5.2 – sleep added to UPROGS in the Makefile

5.2. Build and Test

```
make clean && make qemu
```

The build compiled and linked the new program and added it to the disk image:

```
riscv64-linux-gnu-gcc ... -c -o user/sleep.o user/sleep.c
riscv64-linux-gnu-ld ... -o user/_sleep user/sleep.o user/ulib.o user/usys.o
user/printf.o user/umalloc.o
mkfs/mkfs fs.img README user/_cat ... user/_ls user/_sleep user/_mkdir ...
```

Figure 5.3 – Compiling and linking sleep and adding it to fs.img (compiler flags shortened)

Test in xv6:

```
$ ls
...
ls             2 10 41552
sleep          2 11 34064
mkdir          2 12 34312
...
$ sleep 50
sleep
$ usage: sleep <ticks>
$
```

Figure 5.4 – sleep in the xv6 file system and its two tests

5.3. Interesting Command Output

`sleep` now appears in `ls` as a file of 34064 bytes with inode 11, and every program after it moved down by one inode number (`mkdir` was 11 before, now 12).

The test output also shows how the console works. I typed the second command, `sleep` without a number, while `sleep 50` was still waiting. The characters appeared on screen immediately, because the kernel's console driver echoes input as it arrives. But the shell read them only after `sleep 50` finished, so the `$` prompt and `usage: sleep <ticks>` came afterwards. This shows that the shell waits for its child process to exit before running the next command.

6. Conclusion

In this laboratory work I built xv6 from source with a RISC-V cross-compiler and booted it in QEMU with 3 cores and 128 MB of RAM. Its shell supports the same basic ideas as Linux, such as running programs and connecting them with pipes, with far fewer features.

Reading `cat.c`, `user.h` and `sysfile.c` showed the boundary between user programs and the kernel: a program like `cat` only calls `read` and `write`, and the real work is done by `sys_read` and `sys_write` inside the kernel.

Finally, I wrote my first program for an OS I compiled myself, `sleep`, added it to the build and tested both the normal case and the error case. In this xv6 version the waiting system call is called `pause`, so the program uses `pause(atoi(argv[1]))`.